

Lisbon Airport

This Quarto document lists the data preparatory action for integrating Lisbon airport, LPPT. The APDF data for LPPT was extracted from ECTRL data warehouse and stored locally. This script is not reproducible if the ADPF data are not accessible.

```
library(tidyverse)
```

```
— Attaching core tidyverse packages — tidyverse 2.0.0
—
✓ dplyr      1.1.4      ✓ readr      2.1.5
✓ forcats    1.0.0      ✓ stringr    1.5.1
✓ ggplot2     3.5.1      ✓ tibble     3.2.1
✓ lubridate  1.9.4      ✓ tidyr      1.3.1
✓ purrr       1.0.2
— Conflicts — tidyverse_conflicts()
—
* dplyr::filter() masks stats::filter()
* dplyr::lag()     masks stats::lag()
i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all
  conflicts to become errors
```

```
source(here::here("R","rqutils-zip.R"))
source(here::here("R","rq-apdf-to-punc.R"))
source(here::here("R","rq-apdf-to-pbwg.R")) # todo- clean - need to load
punc first.
source(here::here("03-traffic_characterization-graphs.R"))
```

Attaching package: 'flextable'

The following object is masked from 'package:purrr':

compose

Attaching package: 'zoo'

The following objects are masked from 'package:base':

as.Date, as.Date.numeric

Attaching package: 'magrittr'

The following object is masked from 'package:purrr':

set_names

The following object is masked from 'package:tidyr':

extract

Using poppler version 23.04.0

Loading required package: usethis

Attaching package: 'arrow'

The following object is masked from 'package:magrittr':

is_in

The following object is masked from 'package:lubridate':

duration

The following object is masked from 'package:utils':

timestamp

Rows:	2192	Columns:	3—	Column	specification
-------	------	----------	----	--------	---------------

Delimiter: ","

dbl (2): DLY_FLTS, MVTS_NORM_ROLLAVG

date (1): DATE

i Use `spec()` to retrieve the full column specification for this data.

i Specify the column types or set `show\_col\_types = FALSE` to quiet this message.Rows: 60 Columns: 3— Column specification

Delimiter: ","

chr (1): ICAO

dbl (2): YEAR, TOT_FLTS_YEAR

i Use `spec()` to retrieve the full column specification for this data.

i Specify the column types or set `show\_col\_types = FALSE` to quiet this message.Rows: 716 Columns: 4— Column specification

Delimiter: ","

chr (1): ICAO

dbl (3): YEAR, MONTH, TOT_FLTS_MONTH

i Use `spec()` to retrieve the full column specification for this data.

i Specify the column types or set `show\_col\_types = FALSE` to quiet this message. New names:

```
lppt_zip    <- list.files(pth_apdf, pattern = "^LPPT-apdf")
whats_there <- check_zip_content(pth_apdf, lppt_zip)
whats_there
```

	Name	Length	Date
1	LPPT_01-JAN-2019_31-DEC-2019.gz.parquet	47842469	2025-04-04 15:44:00
2	LPPT_01-JAN-2020_31-DEC-2020.gz.parquet	21018298	2025-04-04 15:43:00
3	LPPT_01-JAN-2021_31-DEC-2021.gz.parquet	26818339	2025-04-04 15:43:00
4	LPPT_01-JAN-2022_31-DEC-2022.gz.parquet	44720672	2025-04-04 15:42:00
5	LPPT_01-JAN-2023_31-DEC-2023.gz.parquet	48624270	2025-04-04 14:05:00
6	LPPT_01-JAN-2024_31-DEC-2024.gz.parquet	47881436	2025-04-04 14:03:00

Airport Traffic

For BRA-EUR, we use PBWG daily traffic stats as analytic data.

```
# let's iterate over each year
whats_there <- whats_there |> mutate(YEAR = str_sub(Name, 13,16))

# loop for one
read_one_year <- function(.pth_apdf, .archive, .yr, ...){
  this_target <- whats_there |> dplyr::filter(YEAR == .yr)
  this_data    <- read_zip(.pth_apdf, .archive, .files = this_target$Name)
  # this_data |> read_and_write_apr_tfc("LPPT", .yr)
}

# test-run: read_one_year(pth_apdf, lppt_zip, .yr = 2019) |> print()
# do all
whats_there$YEAR |> purrr::walk(.f = ~ read_one_year(pth_apdf, lppt_zip, .x))
```

- document write out of traffic (count) data set
- script changed to

Check what we have - traffic data set

```
tfc_data <- list.files(here::here("data-prep-eur"), pattern = "apt-tfc-LPPT",
full.names = T) |>
  purrr::map(.f = ~ readr::read_csv(.x, show_col_types = F))
```

```
lppt_annual_tfc <- tfc_data |> bind_rows() |>
  mutate(YEAR = lubridate::year(DATE)) |>
  group_by(ICA0, YEAR) |>
```

```
reframe(MVTS = sum(ARRS) + sum(DEPS))
```

```
lppt_annual_tfc |> gt::gt()
```

ICAO	YEAR	MVTS
LPPT	2019	221787
LPPT	2020	90080
LPPT	2021	116087
LPPT	2022	203161
LPPT	2023	227381
LPPT	2024	229900

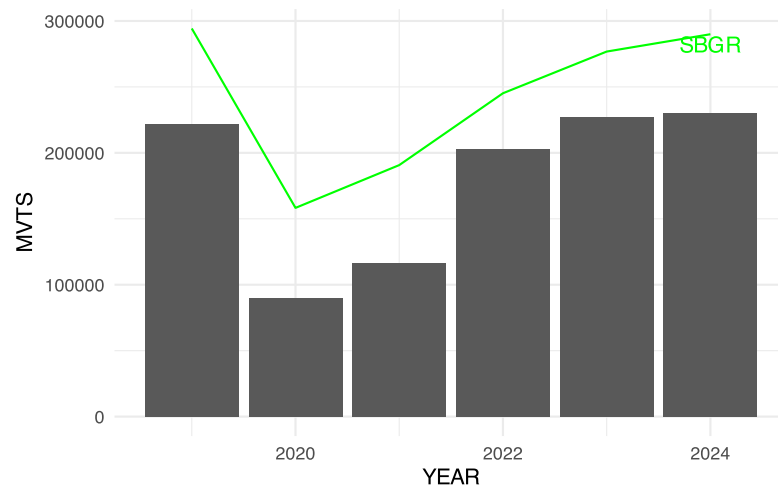
```
tfc_aps_bra <- read_csv("../data/BRA-airport-tfc.csv", show_col_types = FALSE)
```

New names:

- `` -> `...1`

```
sbgr_tfc_annual <- tfc_aps_bra |> filter(ICAO %in% "SBGR") |> mutate(YEAR =  
lubridate::year(DATE)) |> group_by(ICAO, YEAR) |> reframe(MVTS = sum(ARRS) +  
sum(DEPS))
```

```
lppt_annual_tfc |> ggplot() +  
  geom_col(aes(y = MVTS, x = YEAR)) +  
  geom_line(data = sbgr_tfc_annual, aes(y = MVTS, x = YEAR, group = ICAO), color  
= "green" ) +  
  geom_text(  
    data = sbgr_tfc_annual |> filter(YEAR == 2024)  
    , aes(y = MVTS, x = 2024, label = ICAO)  
    , color = "green"  
    , vjust = 1.2 )
```

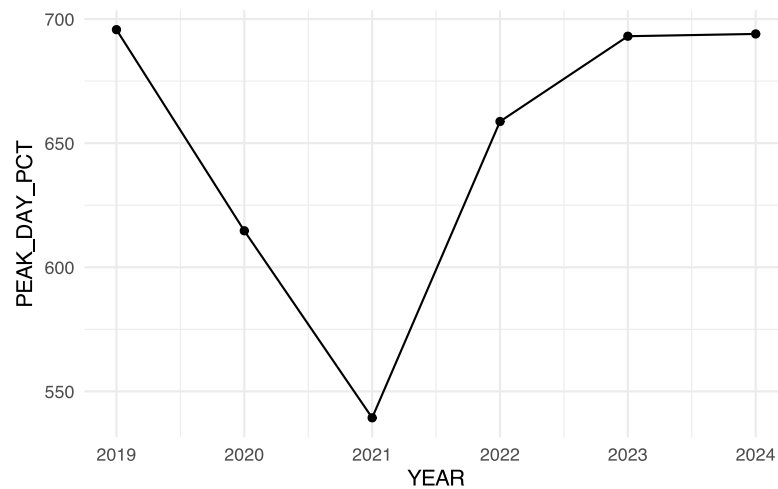


Airport Peak Traffic

based on BRA-EUR, i.e. 99th percentile

```
lppt_annual_peak <- tfc_data |> bind_rows() |> peak_day_from_counts() |>
  add_nbr_rwy() |>
  mutate(REGION = "EUR")

lppt_annual_peak |>
  ggplot(aes(x = YEAR, y = PEAK_DAY_PCT)) +
  geom_line(aes(group = ICAO)) +
  geom_point()
```



Airport Punctuality

```
lppt_adpf <- read_zip(pth_apdf, lppt_zip)
```

```

# do it for one
calc_punc_for_df_with_delay <- function(.df){
  this_punc <- .df |>
  prep_apdf() |>
  distinct() |>
  mutate(ICA0 = "LPPT") |>
  add_delay_and_dlygrp() |>
  package_pbwg_punc_analytic()
  return(this_punc)
}

punc_2019 <- lppt_adpf[[1]] |> calc_punc_for_df_with_delay()

# iterate over all
punc_lppt <- lppt_adpf |>
  purrr::map(.f = ~ .x |> calc_punc_for_df_with_delay())

punc_lppt |> bind_rows() |>
  readr::write_csv(here::here("data", "PBWG-EUR-PUNC-LPPT-2019-2024.csv"))

```